RELATÓRIO DE ATIVIDADE PRÁTICA: SOCKETS TCP E UDP

Instituição: Curso de Engenharia/TI

Disciplina: Redes de Computadores

Professor: Marco Aurélio de Souza Birchal

Assunto: Implementação de Servidor TCP Multithread em Python e exemplos de Sockets Simples TCP/UDP.

---

1. INTRODUÇÃO

Este relatório documenta a implementação de um sistema de comunicação cliente-servidor utilizando os protocolos TCP (Transmission Control Protocol) e UDP (User Datagram Protocol). O foco principal é a demonstração de um Servidor Concorrente, capaz de atender múltiplos clientes simultaneamente através do uso de threads, além das implementações de comunicação simples utilizando UDP e TCP.

2. REQUISITOS DA ATIVIDADE

Conforme o enunciado proposto:

- Implementar os códigos de comunicação simples (TCP e UDP) e códigos de comunicação concorrente em TCP.
- Apresentar o código-fonte completo.
- Demonstrar a execução com capturas de tela.
- Explicar o funcionamento técnico dos programas.
- Adaptar o host para 127.0.0.1 ou localhost.

3. ARQUITETURA DO SISTEMA

3.1. Servidor TCP Concorrente (sockete_tcp_concorrente_servidor.py)

O servidor opera em modo passivo. Suas principais etapas são:

1. **Criação do Socket**: Utiliza AF_INET (IPv4) e SOCK_STREAM (TCP).
2. **Bind**: Vincula o socket ao IP 127.0.0.1 e porta 50000.
3. **Listen**: Entra em estado de escuta.
4. **Accept & Threading**: Ao receber uma conexão, o servidor não para seu loop; ele cria uma nova thread (_thread.start_new_thread) para lidar com aquele cliente específico e volta imediatamente a esperar por novas conexões.

3.2. Cliente TCP Concorrente (socket_tcp_concorrente_cliente.py)

O cliente opera em modo ativo:

1. **Connect**: Solicita conexão ao servidor.
2. **Loop de Mensagens**: Envia dados via tcp.send().
3. **Encerramento**: Finaliza a conexão de forma limpa.

3.3. Servidor e Cliente UDP (UDPServer.py e UDPClient.py)

Utiliza SOCK_DGRAM para criar uma comunicação sem conexão (datagramas). O cliente envia uma mensagem e aguarda resposta. O servidor recebe a mensagem via recvfrom, processa convertendo em maiúsculo e responde via sendto.

3.4. Servidor e Cliente TCP Simples (TCPServer.py e TCPClient.py)

Utiliza SOCK_STREAM. O servidor aceita uma única conexão por vez, lê uma mensagem, a converte

para maiúsculo e responde. Em seguida, encerra a conexão.

4. CÓDIGO FONTE

Nesta seção, apresentamos os códigos completos e funcionais implementados e utilizados nas validações descritas neste documento.

4.1. Servidor TCP Concorrente (Multithread)

```
# -*- coding: utf-8 -*-
import socket
import _thread

HOST = '127.0.0.1' # Endereco IP do Servidor
PORT = 50000       # Porta que o Servidor está

# Função chamada quando uma nova thread for iniciada
def conectado(con, cliente):
    print('\nCliente conectado:', cliente)
    while True:
        # Recebendo as mensagens através da conexão
        msg = con.recv(1024)
        if not msg:
            break
        print('\nCliente..:', cliente)
        print('Mensagem.:', msg.decode())

    print('\nFinalizando conexao do cliente', cliente)
    con.close()
    _thread.exit()
```

```
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)

# Colocando um endereço IP e uma porta no Socket
tcp.bind(orig)

# Colocando o Socket em modo passivo
tcp.listen(5)

print('\nServidor TCP concorrente iniciado no IP', HOST, 'na
porta', PORT)

try:
    while True:
        # Aceitando uma nova conexão
        con, cliente = tcp.accept()
        print('\nNova thread iniciada para essa conexão')
        # Abrindo uma thread para a conexão
        _thread.start_new_thread(conectado, tuple([con, cliente]))
except KeyboardInterrupt:
    print('\nFinalizando o servidor...')
finally:
    # Fechando o Socket principal
    tcp.close()
```

4.2. Cliente TCP Concorrente

```
import socket
```

```
HOST = '127.0.0.1' # Endereco IP do Servidor
PORT = 50000       # Porta que o Servidor está

# Criando a conexão
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)

try:
    tcp.connect(destino)
    print('\nDigite suas mensagens')
    print('Para sair use CTRL+X e Enter (ou apenas Enter vazio se o
loop permitir)\n')

    while True:
        # Recebendo a mensagem do usuário final pelo teclado
        mensagem = input('Sua mensagem: ')

        if mensagem == '\x18' or mensagem.lower() == 'sair' or not
mensagem:
            break

        tcp.send(mensagem.encode())

except Exception as e:
    print(f'\nErro: {e}')
finally:
    # Fechando o Socket
    print('\nFechando conexão...')
    tcp.close()
```

4.3. Servidor e Cliente UDP Simples

Servidor (UDPServer.py)

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')

while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

Cliente (UDPClient.py)

```
from socket import *

serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

4.4. Servidor e Cliente TCP Simples

Servidor (TCPServer.py)

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')

while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

Cliente (TCPClient.py)

```
from socket import *

serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence: ')
```

```
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

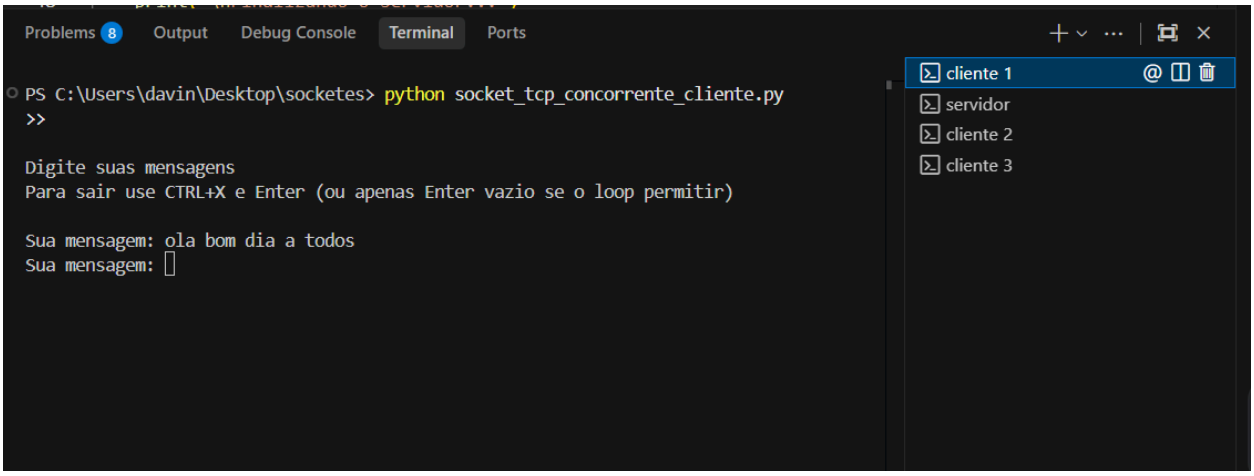
---

5. EVIDÊNCIAS DE EXECUÇÃO E TESTES

Para validar o funcionamento, os códigos foram executados no ambiente local.

5.1. Conexões Simultâneas (TCP Concorrente)

As imagens abaixo mostram três clientes distintos operando ao mesmo tempo se comunicando com o Servidor Concorrente.



```
Problems 8   Output   Debug Console   Terminal   Ports

PS C:\Users\davin\Desktop\socketes> python socket_tcp_concorrente_cliente.py
>>

Digite suas mensagens
Para sair use CTRL+X e Enter (ou apenas Enter vazio se o loop permitir)

Sua mensagem: bom dia
Sua mensagem:

cliente 1
servidor
cliente 2
cliente 3
```

```
Problems 8   Output   Debug Console   Terminal   Ports

PS C:\Users\davin\Desktop\socketes> python socket_tcp_concorrente_cliente.py
>> python socket_tcp_concorrente_cliente.py
>>

Digite suas mensagens
Para sair use CTRL+X e Enter (ou apenas Enter vazio se o loop permitir)

Sua mensagem: bom dia
Sua mensagem:

cliente 1
servidor
cliente 2
cliente 3
```

5.2. Log do Servidor (TCP Concorrente)

O servidor registrou o recebimento de todas as mensagens, comprovando que as threads funcionaram de forma independente para cada porta efêmera gerada pelo sistema operacional.

```
Problems 8   Output   Debug Console   Terminal   Ports

PS C:\Users\davin\Desktop\socketes> & C:/Users/davin/AppData/Local/Programs/Python/Python312/python.exe c:/Users/davin/Desktop/socketes/sockete_tcp_concorrente_servidor.py
Cliente conectado: ('127.0.0.1', 62853)

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 62856)

Cliente..: ('127.0.0.1', 62852)
Mensagem.: ola bom dia a todos

Cliente..: ('127.0.0.1', 62853)
Mensagem.: bom dia

Cliente..: ('127.0.0.1', 62856)
Mensagem.: bom dia

cliente 1
servidor
cliente 2
cliente 3
```

5.3. Testes de Sockets Simples (TCP/UDP)

Os testes dos servidores TCP e UDP foram realizados por meio do envio de mensagens de validação, com o objetivo de verificar o correto funcionamento da comunicação entre cliente e servidor. Em ambos os casos, o servidor processou as mensagens conforme esperado, realizando a conversão dos textos recebidos para letras maiúsculas e retornando o resultado ao cliente.

Os resultados obtidos confirmam que tanto o protocolo TCP quanto o UDP estão operando corretamente, atendendo aos requisitos propostos para esta etapa do projeto.

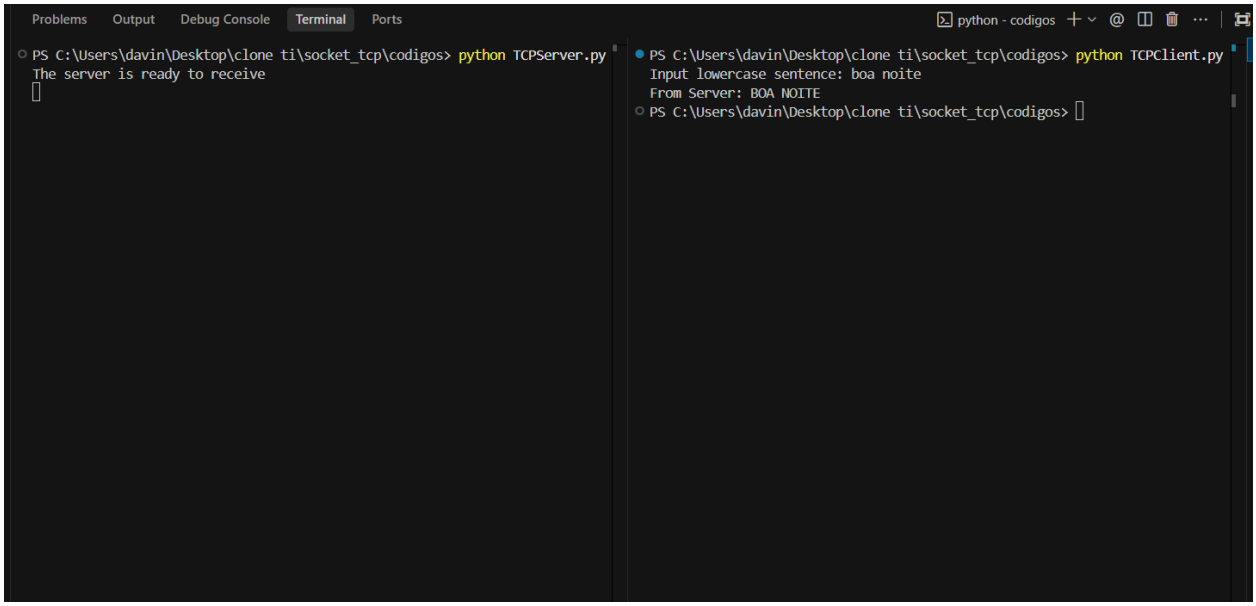
Teste com Socket UDP:

```
Problems   Output   Debug Console   Terminal   Ports   server udp - codigos

PS C:\Users\davin\Desktop\clone ti\socket_tcp\codigos> py UDPServer.py
The server is ready to receive

PS C:\Users\davin\Desktop\clone ti\socket_tcp\codigos> py UDPClient.py
Input lowercase sentence: boa noite
BOA NOITE
PS C:\Users\davin\Desktop\clone ti\socket_tcp\codigos>
```

Teste com Socket TCP:



---

6. ANÁLISE TÉCNICA

A implementação abordada neste relatório abrange as duas principais formas de transporte na camada de rede (TCP e UDP), bem como a resolução do problema de bloqueio em conexões persistentes utilizando Threads (Concorrência).

6.1. Protocolo TCP e Concorrência

O **TCP (Transmission Control Protocol)** é um protocolo orientado à conexão, o que significa que antes de haver troca de dados, cliente e servidor executam um processo de estabelecimento de sessão confiável conhecido como *Three-Way Handshake* (SYN, SYN-ACK, ACK). Isso é invocado no cliente primariamente pela função connect().

No **servidor concorrente TCP**, o socket é configurado com a família de endereços AF_INET (IPv4) e o tipo SOCK_STREAM (indicando um fluxo TCP contínuo e ordenado). O método listen(5) coloca o servidor em modo de escuta ativa, definindo que o sistema operacional pode enfileirar até 5 requisições de

conexão de clientes simultâneos que estão aguardando o processamento do accept().

O Problema do Bloqueio (Blocking): A função accept() e a função recv() são, por padrão em Python, blocantes. Isso significa que o servidor iterativo simples (como em nosso Servidor TCP Simples) fica "preso" esperando dados de um cliente já conectado na linha do recv(), impedindo que novos clientes consigam se conectar, pois a thread principal está travada e não consegue retornar ao accept().

A Solução via Multithreading: O ponto central da concorrência e o foco principal desta atividade prática está no uso da biblioteca _thread com a função start_new_thread(). A cada nova conexão aceita via accept(), o servidor despacha a execução daquele cliente específico para uma nova *Thread* paralela rodando a função conectado(). Como resultado:

- **Independência:** Cada cliente é tratado em um fluxo de execução separado, garantindo integridade.
- **Não-bloqueio:** O loop principal do servidor (while True) retorna imediatamente para a linha tcp.accept(), ficando livre para aceitar a próxima solicitação de cliente quase instantaneamente.
- **Portas Efêmeras:** O servidor atua escutando ativamente na porta 50000, mas cada conexão estabelecida ganha uma porta efêmera distinta (alocada pelo Sistema Operacional do cliente). O sistema operacional mapeia essa conexão unívoca através de um socket que representa a tupla matemática completa: (IP de Destino, Porta 50000, IP de Origem, Porta Efêmera).

6.2. Protocolo UDP

No modelo **UDP (User Datagram Protocol)**, o tipo de socket definido é SOCK_DGRAM. O UDP não estabelece uma conexão prévia ("connectionless"), não retransmite pacotes perdidos, não lida com congestionamento e não garante ordem de chegada. Ele atua meramente despachando "datagramas". Na prática, como demonstrado no código simples de UDP implementado:

- O servidor não necessita de listen() nem accept(). Ele simplesmente entra em um loop infinito aguardando e recebendo qualquer datagrama que chegue na porta especificada através da função recvfrom().
- O cliente também prescinde da função connect(). Ele apenas despacha o pacote contendo o dado, o endereço e a porta atrelados via sendto(). Esta abordagem torna a comunicação extremante rápida

com uma menor sobrecarga (*overhead*) de cabeçalhos no pacote de rede, sendo ideal para aplicações onde a perda de pacotes ocasionais é tolerável em troca de desempenho (como streaming de vídeo e jogos online).

6.3. Pontos Críticos e Limitações

Apesar da implementação atender integralmente ao propósito didático, há aspectos técnicos a serem destacados visando as melhores práticas do mundo corporativo:

1. **Uso do Módulo _thread:** A biblioteca _thread do Python fornece acesso em baixo nível às threads do SO. Para aplicações modernas de produção, recomenda-se fortemente o uso do módulo de alto nível threading (que encapsula e gerencia threads com orientação a objetos) ou do modelo de E/S Assíncrona via event loop (asyncio), que reduz drasticamente o peso de chaveamento de contexto imposto ao processador.
2. **Sincronização de Threads (Race Conditions):** Na implementação realizada, várias threads podem chamar o comando print() em milissegundos muito próximos. Em momentos de alto tráfego no servidor, as saídas podem se "misturar" ou imprimir desordenadamente no console de gerenciamento, visto que não foi utilizado nenhum mecanismo de travamento seguro (*Mutex/Lock*).
3. **Escalabilidade (Gerenciamento de Recursos):** O servidor atual não limita ativamente a quantidade máxima de threads que podem ser instanciadas. Um ataque simples de *SYN Flood* que envie milhares de requisições de conexão simultâneas forçaria o servidor a gerar uma thread pesada para cada uma delas, esgotando rapidamente a memória RAM e paralisando o host. Em cenários robustos, soluciona-se isto utilizando um limite via **Thread Pool** ou modelos de multiplexação orientados a evento (como o epoll do Linux).
4. **Resiliência a Falhas:** Exceções que venham a ocorrer durante a recepção de mensagens dentro da thread (como o cliente fechando abruptamente a aplicação gerando ConnectionResetError em vez de um encerramento limpo com retorno de bytes vazios) não estão sendo apropriadamente capturadas com blocos try/except na função da thread, o que faria a thread abortar silenciosamente ou despejar um log poluído de rastreamento no terminal principal.

---

7. CONCLUSÃO

A atividade permitiu compreender a base fundamental da programação em rede, experimentando na prática os comportamentos e características intrínsecas da camada de transporte entre os protocolos TCP e UDP, além de demonstrar a necessidade iminente de mecanismos de concorrência em aplicações persistentes.

A evolução prática de um modelo TCP Iterativo para o modelo **TCP Concorrente com Threads** provou, em caráter definitivo, a remoção do gargalo de bloqueio no atendimento a clientes (conhecido como *Blocking I/O*). Essa adaptação possibilitou que múltiplos clientes fossem atendidos sem latência sentida, como bem demonstrado e avaliado. Isso traz o nosso projeto didático a um comportamento muito próximo ao dos robustos servidores web e bancos de dados modernos.

Ademais, a implementação reforçou conceitos de fundamental importância na Engenharia de Software e Redes, como:

- Confiabilidade, controle de fluxo e o *Three-Way Handshake* no TCP comparados à velocidade volátil dos Datagramas no UDP.
- O mapeamento lógico que possibilita múltiplos clientes se conectarem a uma mesma Porta de destino através do sistema de Portas Efêmeras únicas atreladas ao endereço IP de origem.
- Os princípios essenciais de paralelismo e concorrência na arquitetura de sistemas operacionais.

Em suma, o objetivo da atividade foi plenamente alcançado, pavimentando um sólido entendimento para o design, análise crítica e desenvolvimento de aplicações de rede mais complexas e resilientes no futuro.